

AgentVerify 2027: Paper Roadmap

Requirement-related failures in coding-agent runs

September 27, 2026

Roadmap

1. Introduction: problem, gap, motivation
2. Data and studies
3. Research questions: RQ1 **in progress**
4. RQ1 progress: what, why, how
5. What we expect from RQ1
6. How RQ1 leads to RQ2 and RQ3
7. Plan for this week and to the deadline

Target: AgentVerify 2027 workshop (co-located with ICSE 2027). Abstract Nov 1, paper Nov 6 (AoE).

The problem: a bridge to the wrong island

Analogy

A builder is asked for a bridge to **Island A**.
The bridge is strong, no cracks, no alarms.
It goes to **Island B**.
Only the **contract** shows the failure.

Real agent run

Task: *“use the Qwen2.5 tokenizer”*
Agent: used **Qwen2**.
No error, no failing command, clean output.
The answer is simply **wrong**.

Monitors that wait for an error, crash or failing test **may never see** this kind of failure.

The gap

- **Failure as a Process** studies *how* coding-agent failures unfold: when the mistake happens, when the run is lost, when a signal first appears.
- FaaP's own monitor experiment hints that requirements matter: giving the monitor the task's requirements raised detection of *specification neglect* from **3% to 22%**.
- But no study, to our knowledge, asks whether **requirement-related failures unfold differently** from other failures, using a requirement label that is **checked against the task text**.

3%→22%: FaaP replication package, docs/oracle_VARIANTS.md. Gap kept modest on purpose.

If requirement failures stay silent more often...

... then waiting for an alarm is not enough.

A monitor must **read the requirement** (the contract) to catch them.

- Early hint in FaaP's own labels: runs labeled *specification neglect* end silent in **44%** of cases, vs **28%** of all failed runs.
- This hint uses FaaP's labels as-is. RQ1 re-tests it with a cleaner, checkable definition.
- Competitor: SWE-RPG: Largest bottleneck: Requirement-related failure 24-46%

	Study 1: FaaP (Terminal-Bench) in progress	Study 2: Nebius (GitHub issues) planned
Runs	1,794 runs (1,184 failed); 89 tasks, 3 agent frameworks, 7 models	~80,000 SWE-agent runs on ~3,600 issues; we sample ~200 failed runs
Requirement	Original task.yaml text, pinned commit	GitHub issue text
Timing labels	FaaP's (mistake, point of no return, first signal)	We create them (LLM draft + human check)
"Silent"	FaaP's label (cannot be re-checked)	A rule on the raw run: any error, traceback or failing test before the end?

Research questions

	Question	Status
RQ1	Do requirement-related failures differ from other failures in when they start , how long they stay fixable , and how often they stay silent ?	in progress
RQ2	Can two people reliably find the step where a run stops following the requirement, using only the requirement text and the run (no test results)?	planned
RQ3	How often do runs stop following the requirement, and does it also happen in runs that pass the tests?	planned

Why we cannot use FaaP's labels directly

FaaP labels every failure **twice**:

- *trigger* (why): `spec_neglect` **176** runs
- *category* (what): names the requirement **251** runs
- **both**: only **116** either: **311**

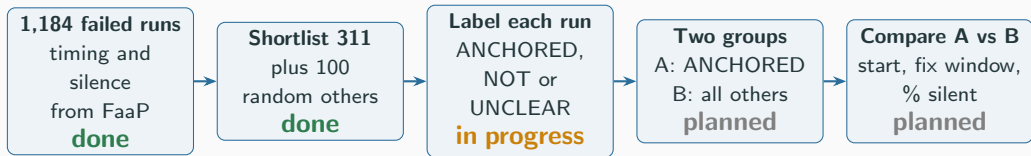
Example: “used Qwen2 instead of Qwen2.5”
is filed as *false premise*.

Requirements not in the text

`rare-mineral-allocation`: FaaP's largest *spec neglect* task (17 runs).
The task text **never** says where to write the answer; only the hidden test expects `/app/solution.txt`.

Like comparing patients with an unreliable diagnosis: **we redo the diagnosis first.**

RQ1 pipeline



- The 100 random runs only **measure** how many requirement failures the shortlist misses; they do not join group A.
- Everything is scripted and reproducible: `scripts/step1_*` to `step3_*`; outputs are identical on rerun.

How one run is labeled

One question

At the decisive mistake, did the agent **stop doing something the task text clearly asked for**?

Yes → ANCHORED **No**, it tried but failed for another reason → NOT **Can't tell** → UNCLEAR

Tie-breaker (the bridge test)

If the agent's own plan had worked perfectly, would the requirement be met?
Plan targets Island B → ANCHORED. Right plan, bad concrete → NOT.

- **Only the task text counts.** Expected only by a hidden test → NOT, marked *implicit*.
- Every ANCHORED label quotes the **exact words** of the task and of the evidence, so anyone can check it.

Two examples

	ANCHORED	NOT
Run	count-dataset-tokens (terminus2, gemini)	create-bucket (miniswe, gpt5)
Task text	"You should use the Qwen2.5-1.5B-Instruct tokenizer"	"Create an S3 bucket named "sample-bucket" using the aws cli"
What happened	Used Qwen/Qwen2-1.5B-Instruct instead	Assumed real AWS credentials were needed; never found the local mock
Bridge test	Plan targets another tokenizer: requirement missed	Plan targets the right bucket: environment belief failed

How we keep it trustworthy

1. **Drafts, not decisions.** Claude drafted every run twice, independently (the two passes agree on 92.7%). Cris decides every label himself.
2. **Second rater, blind.** Kundi labels 50 random runs without seeing any other labels. We report % agreement and Cohen's κ (target ≥ 0.61 ; if lower, fix the guide and relabel).
3. **Freeze before numbers.** The plan is tagged in git before any group comparison. Anything changed later is reported as a change.
4. **Honest limits.** FaaP's timing and silence labels cannot be re-checked (no raw runs released); their per-run agreement file could not confirm their reported agreement.

How we compare: one table, then redraw the tasks

1. One table

	A	B
Median start step	?	?
Median fix window	?	?
% silent	?	?

Gap = A - B, for each row.

2. Is the gap real? Redraw the tasks

1. Put the 87 tasks in a bag, like cards.
2. Draw 87 cards *with replacement* (some twice, some not at all); keep their runs.
3. Recompute the gap.
4. Repeat 10,000 times; keep the middle 95% → the **range**.

Whole range above 0 → the gap does not depend on which tasks happen to be in the benchmark.

Four checks and the decision rule

Check	Doubt it answers	Passes if
Same task	Just harder tasks?	Gap points the same way in more than half of the tasks with ≥ 2 runs in each group
Drop top 5 tasks	Just one or two tasks?	Gap keeps its direction
Every model	Just one bad model?	≥ 5 of 7 models show the same direction
Stricter group	Just our labels?	Gap keeps its direction with FaaP's <code>spec_neglect</code> runs as group A

Decision rule (fixed before we see any numbers)

“Requirement-related failures differ” on a measure **only if** the range excludes 0 **and** all four checks pass. Otherwise: “no clear difference” (also a result). p-value per measure as a footnote (Fisher for % silent, Mann–Whitney for timing)

Step	What	Status
1	Snapshot FaaP data and task text (pinned versions, checksums)	done
2	Runs table; shortlist 311 + 100 random; Kundi's blind 50	done
3	Labels: drafts done; Cris has decided 30 of 411 (the hardest ones first)	in progress
4	Kundi labels 50; agreement script	planned
5	Freeze the plan (git tag)	planned
6	Compare groups: table, redraws, four checks	planned
7	Findings: one sentence + one real example each	planned

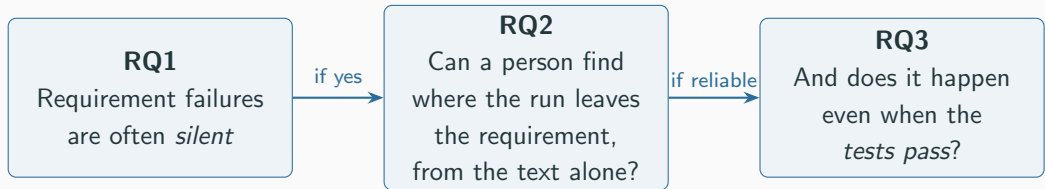
What we expect, and what each outcome means

If we find. . .	It means. . .
Group A more often silent , same start and fix window No clear difference	Requirement failures start like any failure but hide : monitors must read the requirement. Requirement failures are as visible as others; the case for requirement-aware monitors must come from elsewhere.
Nebius agrees with FaaP	The pattern holds across benchmarks and labeling processes.

Useful whatever the outcome

A checked requirement label for FaaP's runs, and a count of FaaP "specification neglect" runs whose requirement is **not in the task text** (explicit vs implicit).

How RQ1 leads to RQ2 and RQ3



- Bridge version: RQ1 shows alarms miss wrong-island bridges; RQ2 asks whether an inspector with the contract can point to the exact spot; RQ3 asks whether bridges that *passed* inspection still go to the wrong island.
- A failing test shows a problem; a passing test does not prove the requirement was met.

This week (Sep 28 – Oct 4)

When	What
Mon	Set up the annotation for Terminal-Bench runs
Mon–Tue	Finish labeling the remaining 381 runs
Mon–Wed	Label 50 runs blind (~1 hour)
Wed	Agreement script; fix guide if needed; freeze the plan
Thu	Run the comparison: table, redraws, four checks
Fri	Write RQ1 findings; pick the Nebius sample and write the silence rule

Road to the deadline

Dates	Milestone
Sep 28 – Oct 4	RQ1 Study 1 (FaaP) done
Oct 5 – Oct 23	Study 2 (Nebius, ~200 runs); RQ2 and RQ3 on Nebius Draw 2
Oct 24 – Nov 1	Write the paper; abstract due Nov 1
Nov 1 – Nov 6	Polish; paper due Nov 6 (AoE) ; full paper, 8 pages incl. references